

Capstone Summary

Python Workflow

1. Python phase: dataset loading

The first phase of the project was carried out in Python using the pandas library.

The original DataCo Supply Chain dataset was imported from a CSV file. During the loading step, an encoding issue appeared, so the file was read using latin1 encoding.

After loading the file, an initial exploration of the dataset was performed to check:

- number of rows and columns;
- column names;
- data types;
- missing values;
- duplicated rows;
- uniqueness of the main identifiers.

A working copy of the dataset was then created, so that the original file could be preserved.

2. Python phase: column renaming and reorganization

The original columns were renamed using snake_case, making them easier to read and easier to use in SQL and Power BI.

The variables were reorganized into logical groups:

- customer information;
- order information;
- delivery location;
- registration or purchase location;
- product and category;
- order item details;
- sales and profit;
- shipping and delivery.

An important decision was to clarify the meaning of the geographic fields. The columns related to Customer City, Customer State, Customer Country, and similar fields were interpreted as information linked to the registration or purchase location, not as the customer's personal address.

For this reason, these fields were renamed as:

- store_city;
- store_state;
- store_country;
- store_zipcode;
- store_latitude;
- store_longitude.

The order-related geographic fields were instead interpreted as information about the delivery destination:

- delivery_city;

- delivery_state;
- delivery_region;
- delivery_country;
- delivery_market.

3. Python phase: removal of unnecessary columns

Some columns were removed because they were not useful for the analysis, were redundant, or were not suitable for the final model.

The removed columns included:

- customer email;
- customer password;
- product link;
- product description;
- store street address;
- delivery zipcode;
- product stock status.

This step helped simplify the dataset and reduce unnecessary information before importing the data into SQL.

4. Python phase: data type correction

Some data types were corrected.

The date columns were converted to datetime format:

- order_date;
- shipping_date.

The store_zipcode column was handled in a way that kept a consistent format compatible with the final export.

Additional checks were then performed on:

- missing values;
- duplicates;
- uniqueness of order_item_id;
- consistency between key columns.

The analysis confirmed that the dataset has order item level granularity. This means that each row represents one single order item.

This was verified because order_item_id is unique and the number of distinct order_item_id values matches the total number of rows.

5. Python phase: validation of relationships between columns

During the checking phase, some key columns were compared to understand how to build the relational model in SQL.

The checks showed that:

- order_item_product_card_id matches product_card_id;
- order_customer_id matches customer_id;
- product_category_id matches category_id.

These checks showed that some columns were redundant, but still useful for building the relationships between the fact table and the dimension tables.

In particular, `product_category_id` was kept in the product table as a foreign key to `dim_category.category_id`.

6. Python phase: correction of data anomalies

Some anomalies were found in the fields related to the store location.

A few rows had missing or inconsistent values in fields such as city, state, and zipcode. These rows were manually corrected using valid combinations already present in the dataset.

The corrections included:

- Elk Grove, CA, zipcode 95758;
- El Monte, CA, zipcode 91732.

After these corrections, the missing values related to `store_zipcode` were resolved.

7. Python phase: text standardization and final export

The text columns were cleaned by removing leading and trailing spaces.

The country label was also standardized:

- EE. UU. was replaced with USA.

At the end of the cleaning process, the final dataset was exported as a CSV file named `dataco_cleaned_final.csv`.

This file was then used as the basis for the import into MySQL.

SQL Workflow

8. SQL phase: database and staging table creation

After the Python cleaning phase, the final dataset `dataco_cleaned_final.csv` was imported into MySQL to build a more structured data model suitable for Power BI.

A dedicated database was created for the DataCo Supply Chain project. A staging table called `dataco_cleaned_final_2` was also created and used as an intermediate table to load the cleaned CSV exported from Python.

The staging table contains all the information needed for the following modeling steps:

- customer data;
- order data;
- product data;
- category data;
- store data;
- delivery location data;
- sales and profit metrics;
- shipping, delivery, and delay data.

The staging table was therefore the starting point for creating the final tables of the model.

9. SQL phase: import of the cleaned CSV

The file `dataco_cleaned_final.csv` was imported into MySQL using the `LOAD DATA INFILE` command.

The import was performed using:

- utf8mb4 character set;
- comma-separated fields;
- text values enclosed in double quotes;

- the first row, containing the headers, skipped during the import.

After the import, a check was performed on the total number of rows and the number of distinct order_item_id values.

The result was:

- total rows: 180,519;
- distinct order_item_id values: 180,519.

This confirmed that the import was successful and that the dataset kept the correct granularity.

10. SQL phase: logical model design

Starting from the staging table, an analytical model was built using dimension tables and a central fact table.

The final model includes:

- dim_category;
- dim_product;
- dim_customer;
- dim_store;
- dim_delivery_location;
- dim_order;
- fact_order_items.

The central table is fact_order_items, because the main level of detail in the dataset is the order item row.

11. SQL phase: creation of dimension and fact tables

The dimension tables were created from the distinct values available in the staging table.

- dim_category contains the product categories, identified by category_id.
- dim_product contains product information, with product_card_id as the primary key and product_category_id as a foreign key to dim_category.
- dim_customer contains customer information, with customer_id as the primary key.
- dim_store contains information about the store or registration/purchase location. Since store_department_id was not unique, a technical key called store_id was created.
- dim_delivery_location contains information about the delivery location. Since there was no unique natural key, a technical key called delivery_location_id was created.
- dim_order contains order-level information, including order_id, order_date, transaction_type, and order_status.

The fact table fact_order_items contains:

- the primary key order_item_id;
- foreign keys to the dimension tables;
- ordered quantity;
- product price;
- discounts;
- order item total;
- sales;
- profit;
- shipping mode;
- scheduled shipping days;
- actual shipping days;

- delivery status;
- late delivery risk.

To populate fact_order_items, INNER JOINS with dim_store and dim_delivery_location were used. These joins were needed to retrieve the technical keys generated in the dimension tables.

The joins were built by comparing the descriptive fields that uniquely identify each store and delivery location.

13. SQL phase: final checks and view creation

After creating the model, final checks were performed on the row count of all tables.

The main check verified that fact_order_items still contained 180,519 rows and 180,519 distinct order_item_id values.

This confirmed that no rows were lost during the transformation from the flat dataset to the relational model.

In a second SQL phase, some views were created for Power BI. The goal was to simplify the connection and make the fields used in the report clearer.

The main view, vw_fact_order_items, enriches the fact table with information from dim_order, such as order date, transaction type, and order status.

Some useful reporting fields were also created in the fact view, including:

- order_date_only;
- shipping_delay_days;
- late_delivery_risk_label.

Descriptive views for product and customer data were also created to make the data easier to use in Power BI.

The final result of the SQL phase is a structured model that is more suitable for BI analysis and can be connected to Power BI through dedicated views.

Power BI Workflow

14. Power BI phase: report and dashboard creation

The model created in SQL was connected to Power BI to build the final dashboard of the project.

The views and tables needed for the report were imported into Power BI:

- dim_product view;
- dim_customer view;
- dim_store;
- dim_delivery_location;
- fact_order_items view.

The Power BI model keeps the same logic created in SQL, with a central fact table at order_item_id level and several connected dimension tables.

A calendar table was created to support time-based analysis and the year filter.

DAX measures were created to calculate the main indicators of the report, including:

- total sales;
- total profit;
- profit margin;
- total orders;

- total customers;
- average order value;
- average spend per customer;
- average orders per customer;
- late delivery rate;
- on-time rate;
- average shipping days;
- average delay days.

An important choice was related to the granularity of the metrics. The dataset is not at single order level, but at order item level: each row represents one product included in an order.

This means that the same order_id can appear more than once when an order contains multiple products. In fact, order_item_id is unique, while order_id can be repeated.

This distinction was necessary because the dataset has order item level granularity, but some KPIs need to be read at order level.

For sales, profit, quantities, products, categories, and discounts, the order item level was kept because each row contributes to the total value of the order.

For order and delivery KPIs, DISTINCTCOUNT(order_id) was used instead, so that each order is counted only once. This avoids overcounting KPIs such as Total Orders, Late Delivery Orders, On-Time Rate, and Late Delivery Rate.

The final dashboard was organized into three main pages:

- Overview;
- Delivery;
- Customer & Store.

The Overview page shows a general summary of performance, including:

- total sales;
- total profit;
- number of orders;
- profit margin;
- late delivery rate;
- average delay;
- annual trend of sales and margin;
- top categories by sales and related profit;
- geographic distribution of store profit.

The Delivery page focuses on shipping and delivery performance, including:

- late delivery rate;
- on-time rate;
- delayed orders;
- average shipping days;
- average delay days;
- total orders;
- performance by shipping mode;
- late delivery rate by country;
- comparison between actual and scheduled shipping days;

- delivery status distribution.

In the Delivery page, conditional formatting was applied to the Late Delivery Rate. When the value is above the 20% threshold, it is highlighted visually to signal a critical issue.

The 20% threshold was used as a visual warning benchmark. It also makes it easier to recognize less critical scenarios when filters are applied.

The Customer & Store page analyzes commercial behavior by customer and store location, including:

- total customers;
- average spend;
- average orders;
- average order value;
- total sales;
- profit margin;
- store city performance;
- sales by payment method;
- sales by customer segment.

To improve the map readability, a concatenated store location field was created by combining city, state, and country.

A classification was also created to distinguish Puerto Rico locations from USA locations. This classification was used as the map legend.

The report includes slicers to filter the data by year and geographic area.

A technical note was added to highlight that 2018 contains partial-year data and should therefore be interpreted with caution when compared with complete years.

A hidden Technical Notes page was also added to document:

- the purpose of the report;
- the page structure;
- the data model;
- the dataset granularity;
- the use of DAX measures and calculated columns;
- the Late Delivery Rate logic;
- the note on partial data for 2018.

The final result is an interactive Power BI dashboard designed to analyze sales, profit, customers, stores, shipping, and late delivery risk, while keeping the SQL model, dataset granularity, and DAX measures consistent.